

TODAYS TOPIC

Why Computerized Data Processing? / Need of RDBMS / Advantages of RDBMS / Why RDBMS

Application of DBMS/RDBMS

Types of data

how to use SSMS (SQL SERVER MANAGEMENT STUDIO)

HOW TO CREATE DATABASE / TABLE

Datatypes in sql server

DDL ,DML,DQL Commands with example

Why Computerized Data Processing? / Need of RDBMS / Advantages of RDBMS / Why RDBMS

Ans:

1. For storing huge amount of data at central place
2. Reduced Application development Time
3. offers data integrity & security
4. easy to store & retrieve data efficiently
5. storing different information at one place
6. For Report generation, Auditing, evaluation
7. Protection from unauthorized access
8. Restrict the wastage of space
9. There is no any data Redundancy
10. There is no any data inconsistency
11. Data must be Recovered from Accidental loss

Application of DBMS/RDBMS

Banking

Airlines

Universities

Credit card transaction

Telecommunication

Finance

Manufacturing

Human Resource

Types of data

Handwritten notes illustrating types of data:

- Unstructured data:** e.g. PPT, Images, videos, letters, memos, researches.
- Semi-structured data:** e.g. XML, JSON.
- Structured data:** Database, RDBMS, Spread sheet.

Examples of data formats:

Unstructured data:

The university has 5600 students. John's ID is number 1, he is 18 years old and already holds a B.Sc. degree. David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semi-structured data:

```
<University>
<Student ID="1">
  <Name>John</Name>
  <Age>18</Age>
  <Degree>B.Sc.</Degree>
</Student>
<Student ID="2">
  <Name>David</Name>
  <Age>31</Age>
  <Degree>Ph.D.</Degree>
</Student>
</University>
```

Structured data:

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

NO SQL; (MONGODB)

WEB API

SQL (RDBMS)SQL SERVER

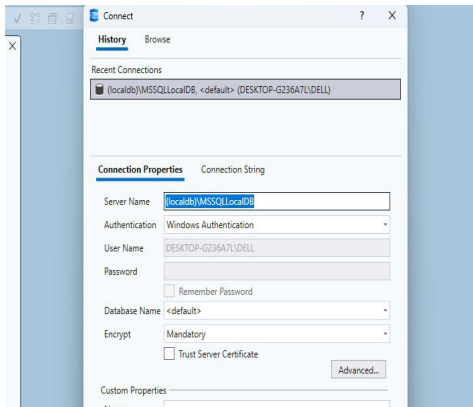
Microsoft SQL Server Installation Guide

Sql Server Management Studio (SSMS)

<https://www.bu.edu/csnet/files/2021/02/SQL-Server-2019-Installation-Guide.pdf>

follow pdf for installation of SSMS

POINTS TO REMEMBER



Database Creation Rules

When initializing a database using either the SSMS GUI

- **Uniqueness:** The database name must be entirely unique
- **Permissions Required:** You must have `CREATE DATABASE` permission or belong to the `sysadmin` or `dbcreator` server roles.
- **File Allocation:** Every database automatically generates at least two files: a **Data File (.mdf)** to house schema/records and a **Log File (.ldf)** for transaction history.

MDF (Master Data File): The primary database file that stores the actual data, including tables, rows, columns, and indexes. This is typically the largest file and its size grows as new data is added to your application.

LDF (Log Data File): The transaction log file that records all database modifications (inserts, updates, deletes) and activities. It is critical for disaster recovery, as it allows SQL Server to restore the database to a consistent state if a crash occurs

Table Creation Rules

Tables act as structured data containers and are strictly governed by structural and data integrity principles.

Permissions Required: You must hold `CREATE TABLE` permissions on the database and `ALTER` permission on the specific schema (such as the default `dbo` schema).

- **Unique Table Names:** No two tables can share an identical name inside the same database schema.
- **Column Definitions:** Every table requires at least **one column**. Each definition must include a **Column Name** and a strict **Data Type** (e.g., `INT`, `VARCHAR (50)`, `DATETIME2`).
- **Nullability Rules:** You should explicitly declare whether a column allows missing data (`NULL`) or mandates a value (`NOT NULL`). Columns default to `NULL` if left unassigned.

- **Comma Separation:** Separate individual column rules with a comma, but ensure the final column in your statement omits the trailing comma.

```
create table test1
(  
  
name1 varchar(50)  
)
```

select this command and click execute.

Then, type

```
insert into test1 values ('Success')
```

select this command and click execute.

SQL Data Types

Choosing the correct data type is important for data integrity, query performance and efficient indexing.

SQL Server Data Types

The data types supported by SQL Server are,

Numeric Data Types

Data Type	Description
BIT	can store single bit (0 or 1) or NULL
TINYINT	can store numbers from 0 to 255
SMALLINT	can store numbers from -32,768 to 32,767
INT	can store numbers between -2,147,483,648 and 2,147,483,647
BIGINT	can store numbers between -9,223,372,036,854,775,808 and 9,223,372,036,854,775,807

String Data Types

Data Type	Description
CHAR(x)	can store characters of fixed length (max 8000 characters)
VARCHAR(x)	can store characters up to given length (max 8000 characters)
TEXT	can store characters up to 2 GB size
IMAGE	can store binary string up to 2 GB size

Date and Time Data Types

DATE	can only store date from January 1, 0001 to December 31, 9999
TIME	can store only time

```
create table customer
(
customerid int
customernumber int
lastname varchar(30) ,
firstname varchar(30)
address varchar(50),
country varchar(50) ,
account_number char(10),

Customergender char(1) , CustomerPostCode char(4) ,
CustomerPhoneNumber char(12)
```

Balance integer

)

real time example 2

```
CREATE TABLE products_catalog
( -- 01. NUMERIC Category
stock_quantity INT,
retail_price DECIMAL(10, 2),
item_weight FLOAT,
discount_rate NUMERIC(4, 2),

-- 02. BINARY Category
product_icon BINARY(16), -- Fixed binary hash
product_manual BLOB, -- Large binary object (e.g., PDF)

-- 03. STRING Category
product_status CHAR(3),      -- Fixed short code (e.g., 'ACT', 'DIS')
product_name VARCHAR(150),   -- Variable length text
description TEXT,            -- Long, unstructured text

-- 04. BOOLEAN Category
is_featured BOOLEAN,

-- 05. DATE/TIME Category
launch_date DATE,
daily_sync_time TIME,
last_updated DATETIME,
-- Extra 06. SPECIAL Data Types (Supported by advanced Relational Databases)
metadata_xml XML,
specifications JSON
product_uuid UUID PRIMARY KEY,

);
```

Real-World Breakdown Matrix			
Category	Data Type	Real-Time Usage Example	Sample Row Value
01. NUMERIC	INT	Warehouse item stock counting	150
	DECIMAL	Financial records / Currency values	299.99
	FLOAT	Scientific/continuous fractional metrics	1.45 (kg)
	NUMERIC	Exact fixed-point percentages	15.50 (%)
02. BINARY	BINARY	Static small cryptography keys or salts	0x3FFA...
	BLOB	Document storage (PDF manuals, raw image files)	0x255044... (Raw Bytes)
03. STRING	CHAR	Standardized status codes	'ACT' (Active)
	VARCHAR	Highly variable names, titles, and headlines	'Pro Noise-Cancelling Headphones'
	TEXT	Unbounded paragraphs, user reviews, or essays	'Experience industry-leading...'
04. BOOLEAN	BOOLEAN	Toggle flags / True or False criteria	TRUE
05. DATE/TIME	DATE	Exact day markers without hour parameters	'2026-06-16'
	TIME	Scheduled repeating batch run times	'04:00:00'
	DATETIME	Audit logging with precision timestamps	'2026-06-16 17:46:00'
06. SPECIAL	UUID	Universally unique system identifier strings	'6ba7b810-9dad-11d1-80b4...'

XML	legacy enterprise configuration feeds	<xml><vendor>AudioCorp...</xml>
JSON	Dynamically structured component properties	{"color": "Matte Black", ...}

```
CREATE TABLE EMPDATA
(
  EID number(4),
  NAME VARCHAR(20),
  PHONE NUMBER(10),
  DOB DATE,
  ADDRESS VARCHAR(20),
  GENDER CHAR,
  SALARY NUMBER(7,2),
  EXPERIANCE NUMBER(2,1)
);
```

```
);
```

```
INSERT INTO EMPDATA VALUES (1000,'PRaViN',5689235698,'01-JAN-2005','pune','m', 10000.58,5.6);
```

DDL :- CREATE ,ALTER, RENAME, DROP , TRUNCATE ,

DML :- INSERT(single/multipal) ,

UPDATE (single/multipal) .

DELETE(single/multipal)

CLAUSES (orderby ,where)

DQL:- SELECT

query for all command

create command use

```
CREATE TABLE teacher
(
  t_id numeric(3,0),
  t_name varchar(20),
  t_class varchar(20),
  t_age numeric(3,1)
);
```

NUMERIC (3,0) : A scale of 0 ensures it acts strictly as a whole integer up to 999. You can also use the **INT** data type if decimal constraints are not strictly required for IDs.

VARCHAR (20) : uses standard **VARCHAR** for variable-length alphanumeric strings

NUMERIC (3,1) : This retains the exact precision requirements. It allows a total of 3 digits, with exactly 2 digits before the decimal point and 1 digit after it (maximum value of 99.9).

run at the end
drop table teacher;

select * from teacher;

-----single insertion for inserting record -----

```
insert into teacher values(1,'riya','first_year',25.6);
select * from teacher;
insert into teacher values(2,'piya','first_year',24.3);
select * from teacher;
insert into teacher values(3,'rajat','first_year',25.6);
select * from teacher;
insert into teacher values(4,'rashmi','third_year',32.5);
insert into teacher values(5,'pranali','first_year',23.6);
insert into teacher values(6,'sonu','first_year',23.3);
insert into teacher values(7,'rani','second_year',24.3);
insert into teacher values(8,'raji','third_year',23.6);
insert into teacher values(9,'priya','second_year',36.2);
insert into teacher values(10,'raj','first_year',25.6);
insert into teacher values(11,'rati','second_year',36.2);
insert into teacher values(12,'raghu','first_year',25.6);
select * from teacher;
```

-----multiple insertion how to insert multiple record in ssms /Execute Bulk Insert Statement-----

```
INSERT INTO teacher (t_id, t_name, t_class, t_age)
VALUES (13, 'Amit Sharma', 'first_year', 35.5), (14, 'Priya Nair', 'second_year', 42.0), (15,
'Rohan Das', 'third_year', 29.5);
```

select * from teacher;

-----selection (SINGLE/COMBINE/MULTIPAL)-----

```
select t_id from teacher;

select t_id,t_name from teacher;

select t_id,t_name,t_class from teacher;

select * from teacher;
```

-----displaying table structure -----

To display a table structure in **SQL Server Management Studio (SSMS)**, the fastest and most efficient methods are using the **ALT + F1 keyboard shortcut** or running the **sp_help system stored procedure**. Note that the standard SQL **DESCRIBE** or **DESC** commands do not work natively in Microsoft SQL Server

```
EXEC sp_help 'teacher';
```

-----alter-----

`ALTER` command primarily as a Data Definition Language (DDL) statement to modify the structure of existing database objects without losing the underlying data

Add a Column

Adds a completely new field to your table definition

Modify an Existing Column

Changes the data type, sizing properties, or nullability settings of a column via `ALTER COLUMN`.

Drop a Column

Permanently deletes a column and all corresponding data from the schema.

```
alter table teacher add loc varchar(20);
select * from teacher;
alter table teacher add t_hobby varchar(20);
select * from teacher;

alter table teacher drop column t_hobby;

select * from teacher;

alter table teacher add t_exp number(3),t_course varchar(20); --not possible
alter table teacher drop column t_exp,t_course; --not possible
select * from teacher;
```

-----rename column name -----

```
alter table teacher add t_hobby varchar(20);
select * from teacher;
EXEC sp_rename 'teacher.t_hobby', 'interest', 'COLUMN';
alter table teacher drop column interest;
select * from teacher;
EXEC sp_rename 'YourTableName.OldColumnName', 'NewColumnName', 'COLUMN';
EXEC sp_rename 'Employees.EmpName', 'FirstName', 'COLUMN';
```

-----individual update-----

```
update teacher set loc='pune' where t_id=1;

update teacher set loc='mumbai' where t_id=2;

update teacher set loc='satara' where t_id=3;

update teacher set loc='mumbai' where t_id=4;

update teacher set loc='nasik' where t_id=5;

update teacher set loc='mumbai' where t_id=6;

update teacher set loc='nasik' where t_id=7;
```

```

update teacher set loc='satara' where t_id=8;

update teacher set loc='mumbai' where t_id=9;

update teacher set loc='mumbai' where t_id=10;

update teacher set loc='nasik' where t_id=11;

update teacher set loc='nasik' where t_id=12;

update teacher set loc='mumbai' where t_id=13;

update teacher set loc='nasik' where t_id=14;

update teacher set loc='nasik' where t_id=15;

```

```
select * from teacher
```

-----group update -----

```
alter table teacher add school_name varchar(20);
```

```
update teacher set school_name='MPONLINE SCHOOL';
select * from teacher
```

```
-----
alter table teacher add t_salary numeric(7,2);
select * from teacher
```

```

update teacher set t_salary=52369.89 where t_id=1;
update teacher set t_salary=62369.81 where t_id=2;
update teacher set t_salary=12369.82 where t_id=3;
update teacher set t_salary=22369.83 where t_id=4;
update teacher set t_salary=32369.84 where t_id=5;
update teacher set t_salary=82369.85 where t_id=6;
update teacher set t_salary=42369.81 where t_id=7;
update teacher set t_salary=52869.82 where t_id=8;
update teacher set t_salary=58369.83 where t_id=9;
update teacher set t_salary=54369.84 where t_id=10;
update teacher set t_salary=52169.85 where t_id=11;
update teacher set t_salary=57369.86 where t_id=12;

```

```

update teacher set t_salary=54369.84 where t_id=13;
update teacher set t_salary=52964.85 where t_id=14;
update teacher set t_salary=57369.86 where t_id=15;
select * from teacher

```

-----column alias-----

In SQL Server Management Studio (SSMS), a **column alias** is a temporary name given to a column or an expression in your query results. It changes the column header in the output grid but does not change the actual table structure in your database.

You can assign a column alias in three different ways within a `SELECT` statement:

- **Using the `AS` keyword (Recommended):** Explicitly defines the alias, making code highly readable.
- **Using a space:** Omits `AS` and separates the column and alias by a space.

```
select t_salary AS sal from teacher
```

```
select t_salary AS sal ,loc AS loc from teacher
```

```
select t_salary as "sal" from teacher
select t_salary as sal from teacher
```

```
select t_salary AS "sal" from teacher
```

```
select t_salary sal from teacher
```

```
select t_salary sal ,loc location from teacher
```

```
select t_salary*12 anaualsalary from teacher
```

-----arithmetic operator -----

```
select t_name,t_salary,t_salary+100 finalsal from teacher
select t_name,t_salary,t_salary-100 finalsal from teacher
select t_name,t_salary,t_salary*100 finalsal from teacher
select t_name,t_salary,t_salary/100 finalsal from teacher
```

-----concatenation operator-----

```
select t_name || t_class AS "TEACHER DATA" from teacher
```

-----USING literal character string -----

```
select t_name || ' teaching to ' || t_class
```

```
AS "TEACHER DATA"
```

```
from teacher
```

-----distinct -----

What is DISTINCT?

- DISTINCT is a keyword
- is used in conjunction with the `SELECT` statement
- used to eliminate all the duplicate records
- and used fetching only unique records.
- DISTINCT statement is used to return only distinct (different) values.

```
select distinct t_class from teachers
select distinct t_age from teacher;
```

```

select distinct t_class from teacher;

select distinct t_class,loc from teacher;

select distinct t_class,t_name from teacher;

select distinct t_age from teacher order by t_age;

select t_name from teacher order by t_name desc;

```

```

-----order by clause(default ascending)-----
select * from teacher order by t_id desc;
select * from teacher order by t_age desc;
select * from teacher order by t_salary desc;
select * from teacher order by t_id;
select * from teacher order by t_age;
select * from teacher order by t_salary;

```

```

select distinct t_age from teacher order by t_age;

select t_name from teacher order by t_name desc

```

```

---concept for WHERE clause and different types of operator -----

```

```

select t_id,t_name from teacher where t_id=7;
select t_id,t_name from teacher where t_id!=6;
select t_id,t_name from teacher where t_id<>6;
select t_name from teacher where t_age<30;
select t_name from teacher where t_age<=30;
select t_name from teacher where t_age>20;
select t_name from teacher where t_age>=20;
select t_name,t_age from teacher where t_age between 20 and 30;
select t_name,t_age from teacher where t_age in (23.6,25.6);
select t_name,t_age from teacher where t_age not in (23.6,25.6);

```

```

-----aggregate function -----

```

In SQL Server Management Studio (SSMS), **aggregate functions perform a calculation on a set of values and return a single scalar value.** They process multiple rows of data across columns to generate summary statistics like totals, averages, or counts.

SUM() : Calculates the total sum of a numeric column.

AVG() : Computes the mathematical average of a numeric column.

COUNT() : Counts the number of rows or non-null values in a set.

MAX() : Identifies the highest value in a column.

MIN() : Identifies the lowest value in a column.

```

select count(*) as total_teachers from teacher;

select sum(t_salary) as total_salary from teacher;

select avg(t_salary) as "Average Salary" from teacher;

select min(t_salary) as "Lowest Salary" from teacher;

```

```
select max(t_salary) as "Highest Salary" from teacher;
```

-----like operator-----

What is Like ?

- Like is a logical operator that is used to determine whether a specific character string matches a specified pattern.
- This operator can be useful in cases when we need to perform pattern matching instead of equal or not equal.
- There are two wildcards used in conjunction with the LIKE operator.
- The percent sign (%)
- The underscore (_)

```
select * from teacher where t_salary like '25000';
```

```
select * from teacher where t_name like '_ _a%';
```

```
select * from teacher where t_name like '%t';
```

```
select * from teacher where t_name like '%a%';
```

extra example

example :-

- **WHERE SALARY LIKE '200%'**
 - Finds any values that start with 200.
- **WHERE SALARY LIKE '%200%'**
 - Finds any values that have 200 in any position.
- **WHERE SALARY LIKE '2__3'**
 - Finds any values in a five-digit number that start with 2 and end with 3.
- **WHERE SALARY LIKE '_2%3'**
 - Finds any values that have a 2 in the second position and end with a 3.
- **WHERE SALARY LIKE '%2'**
 - Finds any values that end with 2.

-----rename-----

```
EXEC sp_rename 'teacher', 'instructor';
select * from teacher
select * from instructor
```

-----drop-----

```
drop table instructor
select * from instructor
EXEC sp_help 'instructor';
```

-----truncate-----

```
truncate table givetablename
select * from givetablename
EXEC sp_help 'givetablename'
```

-----working with data date data type-----

```
create table employee
(
name varchar(20) ,
occupation varchar(20) ,
working_date date,
working_hours numeric(10),
salary numeric(7)
);
select * from employee
```

```
insert into employee values('Jolly Evans','HR','4-Oct-2020',9,25000);
select * from employee
```

```
insert into employee values('Laura Paul','manager','4-oct-2020',10,45000478); ----not
possible
drop table employee;
```

DELETE	TRUNCATE
It is a DML command.	It is a DDL command.
Used to remove specific rows or all the rows depending upon our requirement.	Can only be used to remove all the rows from a table.
It can use the WHERE clause.	It can't use the WHERE clause.
It is comparatively slower.	It is faster.
It records the deleted rows in the transactions log.	Since it removes the page where our table's data is stored, it stores the page in the transactional log.

S.NO	DROP	TRUNCATE
1.	The DROP command is used to remove table definition and its contents.	Whereas the TRUNCATE command is used to delete all the rows from the table.
2.	In the DROP command, table space is freed from memory.	While the TRUNCATE command does not free the table space from memory.
3.	DROP is a DDL (Data Definition Language) command.	Whereas the TRUNCATE is also a DDL (Data Definition Language) command.
4.	In the DROP command, view of table does not exist.	While in this command, view of table exist.
5.	In the DROP command, integrity constraints will be removed.	While in this command, integrity constraints will not be removed.
6.	In the DROP command, undo space is not used.	While in this command, undo space is used but less than DELETE.
7.	The DROP command is quick to perform but gives rise to complications.	While this command is faster than DROP.

What is SQL?

- SQL stands for Structured Query Language
- SQL lets you access and manipulate databases
- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

Rules for SQL:

- A ';' is used to end SQL statements.
- Statements may be split across lines but keywords may not.
- Identifiers, operator names, literals are separated by one or more spaces or other delimiters.
- A comma(,) separates parameters without a clause.
- A space separates a clause.
- Reserved words can not be used as identifiers unless enclosed with double quotes.
- Identifiers can contain up to 30 characters.
- Identifiers must start with an alphabetic character.
- Characters and date literals must be enclosed within single quotes.
- Numeric literals can be represented by simple values.
- Comments may be enclosed between /* and */ symbols and maybe multi-line.

WHAT IS TCL(DBA)

- TCL is abbreviation of Transactional Control Language.
- It is used to manage different transactions occurring within a database.
- **COMMIT** – Saves work done in transactions
- **ROLLBACK** – Restores database to original state since the last COMMIT command in transactions
- **SAVE TRANSACTION** – Sets a savepoint within a transaction

- **what is DCL(DBA)**
- DCL is abbreviation of Data Control Language.
- It is used to create roles, permissions, and referential integrity as well it is used to control access to database by securing it.
- **REVOKE** – Withdraws user's access privileges to database given with the GRANT command
- **grant** ; - gives users access privilege to database

BETWEEN The BETWEEN operator is used to fetch rows based on a range of values.

IN The IN operator is used to check for values contained in specific sets.

What is the use of 'WHERE' Clause?

WHERE keyword is used for fetching filtered data in a result set. - It is used to fetch data according to a particular criteria. - WHERE keyword can also be used to filter data by matching patterns.

What are SQL Commands? Explain types of SQL Commands

These SQL commands are mainly categorized into four categories as:

1. DDL - Data Definition Language
2. DQL - Data Query Language
3. DML - Data Manipulation Language
4. DCL - Data Control Language

1. DDL(Data Definition Language): DDL or Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database. **CREATE** - is used to create the database or its objects (like table, index, function, views, store procedure, and triggers).

DROP - is used to delete objects from the database.

ALTER -is used to alter the structure of the database.

TRUNCATE—is used to remove all records from a table, including all spaces allocated for the records are removed.

RENAME—is used to rename an object existing in the database.

2. DQL (Data Query Language) :

SELECT – is used to retrieve data from the database.

3.DML statements are used for performing queries on the data within schema objects.

INSERT – is used to insert data into a table.

UPDATE - is used to update existing data within a table.

DELETE – is used to delete records from a database table.

TASK FOR HOME

Consider Employee table

EMPNO	EMP_NAME	DEPT	SALARY	DOJ	BRANCH
E101	Amit	Production	45000	12-Mar-00	Bangalore
E102	Amit	HR	70000	03-Jul-02	Bangalore
E103	sunita	Management	120000	11-Jan-01	mysore
E105	sunita	IT	67000	01-Aug-01	mysore
E106	mahesh	Civil	145000	20-Sep-03	Mumbai

Perform the following

1. Display all the fields of employee table
2. Retrieve employee number and their salary
3. Retrieve average salary of all employee
4. Retrieve number of employee
5. Retrieve distinct number of employee
6. Retrieve total salary of employee group by employee name and count similar names
7. Retrieve total salary of employee which is greater than >120000
8. Display name of employee in descending order
9. Display details of employee whose name is AMIT and salary greater than 50000